

1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?

Raw audio is 1d time waveform, and it shows amplitude changes over time. For keyword spotting, the model needs to understand which frequencies are present and how they changed. Spectrogram/MFCC-style features convert audio into a time-frequency representation, which is easier to distinguish words.

2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?

The representative dataset can give examples of realistic input values. The TensorFlow Lite converter uses these samples to decide the proper scale and zero-point for converting floating-point activations into INT8 values. The good representative dataset helps the quantized model behave similarly to the original float model, while it's more compact and faster.

3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.

It loads a batch of audio examples from the speech commands dataset and converts them into the same preprocessed feature format used by the model. It can pull data from training, validation, or testing sets depending on the arguments. It also applies noise mixing and time shifting.

4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?

`G_model` is the TensorFlow Lite model stored as a C/C++ byte array. It contains actual quantized neural network model data, which Arduino code can load.

G_model_len is the size of the model array in bytes.

5. Please include a screenshot showing the correct identification of at least one keyword, either **"YES"** or **"NO"**, as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.

```
16:28:41.452 -> Heard unknown (202) @23824ms
16:28:42.987 -> Heard unknown (228) @25344ms
16:28:45.220 -> Heard unknown (207) @27584ms
16:28:48.991 -> Heard unknown (204) @31360ms
16:28:50.520 -> Heard unknown (205) @32880ms
16:28:56.715 -> Heard unknown (206) @39072ms
16:29:00.036 -> Heard unknown (202) @42400ms
16:29:02.470 -> Heard unknown (201) @44816ms
16:29:11.703 -> Heard unknown (212) @54064ms
16:29:15.004 -> Heard yes (204) @57376ms
16:29:17.069 -> Heard unknown (209) @59440ms
16:29:23.453 -> Heard unknown (215) @65824ms
16:29:24.977 -> Heard unknown (225) @67344ms
16:29:28.303 -> Heard no (210) @70672ms
16:29:29.670 -> Heard unknown (204) @72016ms
16:29:31.621 -> Heard unknown (201) @73984ms
16:29:33.152 -> Heard unknown (220) @75520ms
16:29:35.388 -> Heard unknown (211) @77760ms
16:29:37.552 -> Heard unknown (205) @79920ms
16:29:39.087 -> Heard unknown (240) @81440ms
16:29:40.621 -> Heard unknown (248) @82976ms
```